

Exploring Residual Connection Variants in Modern CNN Architectures

Emily Moberly
Mu Chen
Maximilian Garcia
Napoleon Salazar

March 2026

1 Introduction

The architectural evolution of convolutional neural networks has historically shown that increasing depth expands representational capacity — but this does not always hold. As established by the paper by Deep Residual Networks (2015), plain sequential networks exhibit a contradictory behavior in which training accuracy degrades as depth increases. This degradation comes from a fundamental optimization failure. In purely sequential architectures, solvers struggle to approximate identity mappings through successive non-linear layers and parameterized weight matrices, causing gradient starvation in earlier layers and preventing effective weight updates.

The residual learning framework addresses this by reformulating the learning objective. Rather than requiring stacked layers to directly fit a desired mapping $H(x)$, they are instead tasked with approximating a residual function $F(x) := H(x) - x$, recasting the complete mapping as $y = F(x) + x$ via a skip connection. As noted by Deep Residual Networks (2015)[3], if the optimal function is closer to an identity mapping than to a zero mapping, it is far easier for the optimizer to drive $F(x)$ toward zero than to calculate the identity function from scratch through multiple non-linear layers.

This study contributes a controlled experimental framework for evaluating four residual routing strategies using CIFAR-10 [6]. All four variants share identical block structures (Conv2d -64 filters, 3×3 , BatchNorm, ReLU, a second Conv2d, and BatchNorm) and are trained under the same protocol (SGD with a StepLR scheduler over 200 epochs). Then, it builds an empirical comparison by recording training and validation accuracies alongside layer-wise L2 gradient norms across 4-layer and 8-layer configurations. The result should clearly show how well each residual strategy addresses the degradation problem, while also revealing whether the added complexity of the Scaled or Gated models actually leads to better performance than the simpler Baseline residual network.

2 Problem Statement

Although the standard skip connection works well, it is just one of several ways to route information through a residual network. The residual contribution can be modified to control information flow differently — for example, by applying a learnable scalar or a data-dependent gate. This raises two questions: first, **how well does each residual variant solve the degradation problem compared to a plain network ?**; and second, **do the more complex Scaled and Gated variants actually perform better than the simpler Baseline skip connection?**.

3 Related Work

The evolution of convolutional neural networks has been driven by the pursuit of deeper architectures. However, increasing depth introduces significant optimization challenges, most notably the degradation problem and vanishing gradients. The following review examines the foundational literature that has shaped modern approaches to deep network optimization, covering key advances in network architecture, normalization techniques, and adaptive routing mechanisms.

3.1 Identifying the Theoretical Barrier to Deep Optimization: The Vanishing Gradient Problem [4]

The motivation for structural modifications in deep architectures traces back to early theoretical work on optimization failures. Hochreiter [4] formally showed that error gradients propagating backward through sequential networks decay exponentially, a phenomenon that prevents earlier layers from updating their weights effectively. This vanishing gradient effect fundamentally limits both the trainable depth and the learning capacity of plain neural networks.

3.2 Establishing Depth as a Prerequisite for Representational Power: VGG Networks [8]

Despite known optimization barriers, increasing depth remained central to advancing visual recognition. The VGG architecture [8] demonstrated that stacking 3×3 convolutional filters allows deeper networks to achieve substantial gains in image classification accuracy while keeping computational complexity manageable. This milestone established depth as the standard design paradigm and exposed the critical need for better training stabilization techniques.

3.3 Stabilizing Gradient Flow via Internal Normalization: Batch Normalization [5]

Batch Normalization [5] was introduced to address the optimization bottlenecks of scaling depth. By normalizing layer inputs across a mini-batch during training, internal covariate shift is reduced, gradient propagation improves, and higher learning rates become viable. It quickly became a standard component of modern deep learning architectures, including residual networks.

3.4 Pioneering Gated Routing for Deep Optimization: Highway Networks [9]

Similarly, structural routing modifications emerged to address the disappearance of gradients at the architectural level. Highway Networks [9] introduced gated shortcut connections capable of dynamically regulating the proportion of an input tensor that is transformed by a layer versus carried forward unaltered. By demonstrating that networks comprising hundreds of layers could be successfully optimized via gradient descent using these pathways, this architecture served as a direct conceptual precursor to the gated and baseline residual mechanisms evaluated in this study.

3.5 Importance of Dynamically Weighting Feature: Attention Mechanisms [1]

Attention mechanisms [1] extended the principle of adaptive weighting to sequence models, allowing networks to dynamically focus on relevant input features rather than compressing the entire input into a static vector. This data-dependent routing significantly improved performance in sequential tasks and laid the theoretical foundations for parameterized residual routing strategies, such as the gated residual variant explored in this study.

3.6 Generalizing Data-Dependent Routing Mechanisms: The Transformer Architecture [10]

The Transformer architecture [10] further cemented the utility of dynamic information routing by replacing convolutional structures entirely with self-attention mechanisms, allowing long-range dependency modeling and highly parallelizable training. Although originally designed for natural language processing, its reliance on adaptive, data-dependent routing highlights the broader relevance of the scaled and gated residual pathways evaluated in this study.

3.7 Adapting Residual Connections for Computational Efficiency: MobileNetV2 [7]

The enduring importance of skip connections is further evidenced by MobileNetV2 [7], which combines depthwise separable convolutions with inverted residual bottleneck layers to maintain strong gradient flow and accuracy while significantly reducing computational cost. This shows that residual-style connections remain a cornerstone of scalable network design, even in architectures optimized for constrained environments.

In synthesis, the reviewed literature traces a clear progression: from the early identification of gradient-based optimization failures to the development of structural and data-dependent routing mechanisms designed to overcome them. Although techniques such as adaptive gating, batch normalization, and residual connectivity are now standard across modern architectures, their specific routing mechanics are rarely isolated and compared under uniform conditions. Building on these theoretical foundations, this study aims to systematically quantify how specific variations in the residual skip connection affect the optimization and performance of deep convolutional networks.

4 No Residual Network / Ablation Study

The plain network variant is a purely sequential stack of convolutional blocks with all skip connections removed. Each block processes an input tensor x through a shared pathway — Conv2d (64 filters, 3×3), BatchNorm, ReLU, a second Conv2d, and a final BatchNorm — producing output $F(x)$. Unlike the residual network architecture, the plain network sets $y = F(x)$ directly, with no shortcut of any kind. The output y is then passed through a final ReLU before being forwarded to the next block. This architecture serves as a controlled baseline, designed to isolate and expose the degradation problem identified by Deep Residual Learning (2015) [3].

In this study, a single variable is isolated: the presence or absence of the $+x$ skip connection. Two depth configurations — 4-layer and 8-layer — are evaluated to determine whether the degradation problem manifests once the skip connection is removed. Since a deeper network is theoretically expected to perform at least as well as its shallower counterpart, any observed decrease in accuracy with increased depth is taken as evidence of an optimization failure. The direct comparison between the two plain network depths is therefore used to establish a localized proof of the degradation problem. Let us analyze the different possible outcomes

4.1 Accuracy Comparison (4-layer vs. 8-layer)

Theoretically, a deeper model should be no worse than a shallower one, as the additional layers could simply be initialized to act as identity mappings, leaving the learned function unchanged. Hence, a deeper (8-layer) plain network is expected to achieve significantly lower top-1 classification accuracy on CIFAR-10 than its shallower counterpart (4-layer). In practice, however, identity mappings are difficult to approximate through multiple non-linear activations and parameterized weight matrices using optimizers such as SGD. Without a skip connection to provide structural assistance, the identity function must be learned entirely through the convolutional layers. Therefore, added depth is expected to impede rather than improve optimization, manifesting itself as a degraded classification accuracy in the 8-layer configuration.

4.2 Training Curve Analysis

During the 200-epoch training period, the 8-layer plain network is expected to maintain a training error consistently higher than that of the 4-layer network. This is a key diagnostic signal: elevated training error confirms that the accuracy drop derives from an optimization failure — the model’s inability to minimize loss — rather than overfitting, which would instead present as near-zero training error with rising validation error.

4.3 Gradient Norm vs. Layer Depth

To understand the cause of the accuracy degradation described above, the gradient flow through both depth configurations is analyzed. As error signals propagate backward through the network, the L2 gradient norms are expected to decay progressively toward earlier layers. In a purely sequential architecture with no skip connections, each backward step multiplies the signal by weight matrices, BatchNorm variance adjustments, and ReLU derivatives — compounding with each layer until gradient magnitudes approach zero. When plotted against the depth of the layer in Epochs 1, 50 and 150, the gradient norm curves are expected to show a decline, with the earliest layers receiving near-zero learning signals throughout training. This gradient starvation prevents the early layers from updating their weights, stopping feature extraction, and directly exposing the degradation problem.

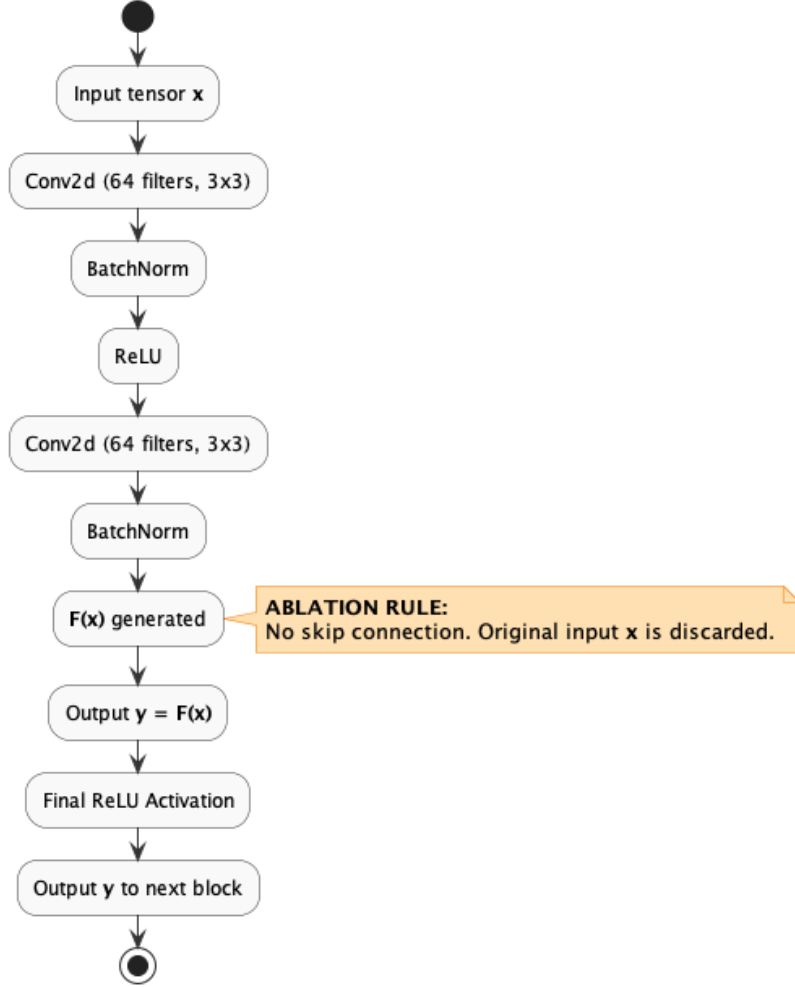


Figure 1: **Plain Network.**

5 Proposed Solutions

5.1 Baseline Residual

The baseline architecture used in this project follows the standard residual block design in the ResNet paper [3]. Residual networks incorporate skip connections that allow the input of a layer to bypass intermediate transformations. The input is directly added to the output of a residual function. This design helps improve gradient flow during backpropagation and reduces optimization difficulties that often exist in deep neural networks.

In our implementation, each residual block consists of a sequence of convo-

lutional layers combined with normalization and nonlinear activation functions. Each block applies two convolutional layers with batch normalization and ReLU activation. The input to the block is then added to the output of the convolutional layers through an identity connection. This addition enables the network to preserve the original input signal easily and learns only the residual function required to represent the complete transformation.

The baseline residual model serves as a reference point for evaluating the other residual variants in this project. All other models are implemented with the same overall network structure, with the only difference on the formulation of the residual connection. This makes sure that any observed differences in training can be attributed independently to changes in the residual connection itself.

The standard residual block can be expressed as

$$y = x + F(x) \tag{1}$$

where x represents the input to the residual block and $F(x)$ denotes the residual function implemented by the stacked convolutional layers within the block. The output y is obtained by adding the input x to the learned residual transformation $F(x)$.

In this configuration, the network does not attempt to learn the full mapping $H(x)$ directly. Instead, it learns the residual component $F(x) = H(x) - x$, which represents the difference between the complete transformation and the identity mapping. This design speeds up the optimization process and allows deeper networks to converge more efficiently. In addition to that, the identity skip connection also provides a direct path for gradient signals to propagate during backpropagation which helps maintain the gradient flow and stable learning even for very deep networks.

The baseline model is expected to demonstrate efficient model convergence due to the presence of identity skip connections. These connections provide a direct pathway for gradient signals to flow and a good pre-conditioning of the transformation, helping maintain effective parameter updates even in deeper architectures. The metrics obtained from the baseline model training process hence serve as a reference for comparing the optimization behavior of different residual variants in this study.

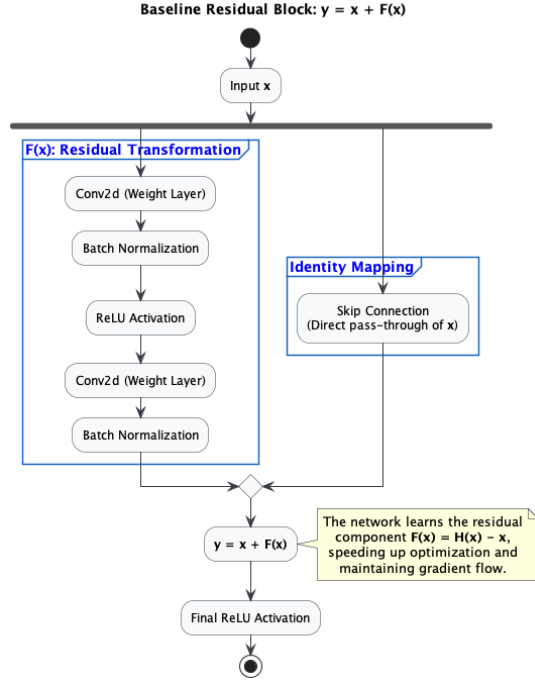


Figure 2: **Baseline Residual Network.**

5.2 Scaled Residual

Although residual skip connections have been highly effective at addressing the degradation problem and improving gradient flow in deep neural networks, the standard formulation assumes that the residual contribution is always added at full strength. In practice, however, the usefulness of the residual update may vary across layers and throughout different stages of training. This suggests that allowing the network to regulate the magnitude of the residual signal could provide additional flexibility during optimization.

The scaled residual variant introduces a learnable scaling parameter α into the standard residual connection to regulate the influence of the residual branch during training. In the early stages of optimization, the features produced by the residual pathway $F(x)$ may be unstable or poorly calibrated. Directly adding these early representations to the identity shortcut can sometimes introduce noise into the network’s internal feature representations.

To address this, the scaled residual formulation applies a learnable scaling factor to the residual branch. Conceptually, this parameter acts as a volume control, allowing the model to adjust how strongly the learned transformation contributes to the final output of the block. By modulating this contribution, the network can gradually increase its reliance on the residual pathway as the convolutional layers begin to learn more meaningful feature representations.

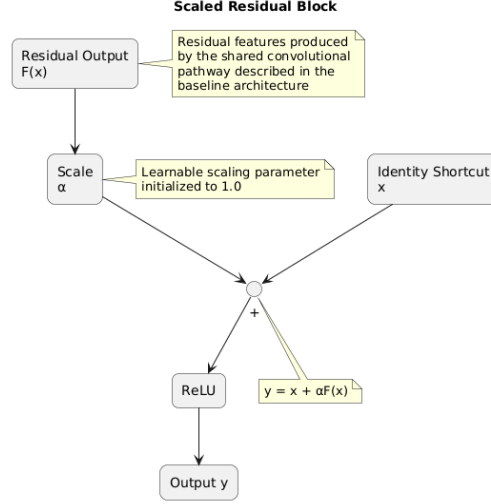


Figure 3: **Scaled residual block.** The residual output $F(x)$ is multiplied by a learnable scaling parameter α before being added to the identity shortcut x . The shared convolutional pathway that produces $F(x)$ is illustrated in Figure ??.

Importantly, this variant preserves the exact same block structure used in the baseline residual network. Each block contains two 3×3 convolutional layers with 64 filters, followed by batch normalization and ReLU activations. By keeping the architectural components identical, any differences in training behavior or performance can be attributed solely to the modified residual routing mechanism rather than other structural changes.

The scaled residual block can be expressed as

$$y = x + \alpha F(x) \quad (2)$$

where x represents the input tensor entering the block and $F(x)$ denotes the transformation produced by the convolutional pathway. The parameter α is a learnable scalar initialized to 1.0 so that the network begins training with behavior equivalent to the standard residual formulation. During training, gradient-based optimization allows the model to adjust the value of α , enabling it to regulate the magnitude of the residual contribution.

The primary hypothesis behind this variant is that introducing a learnable scaling parameter may improve training stability and optimization behavior compared to both plain sequential networks and fixed-strength residual connections. Early in training, the model may benefit from relying more heavily on the identity shortcut x , while gradually increasing the contribution of the learned residual transformation as feature representations improve.

The learned value of α may also provide insight into how the model balances identity preservation and feature transformation across layers. Values of $\alpha < 1$

would indicate that the network prefers smaller residual updates, suggesting that full-strength residual additions may be overly aggressive in certain layers. Conversely, values of $\alpha > 1$ would imply that stronger residual contributions help the model refine its representations.

Crucially, the identity shortcut x remains unchanged and continues to provide a direct pathway for gradient propagation. As a result, the scaled residual architecture is expected to maintain the healthy gradient flow characteristic of residual networks while providing additional flexibility in how residual updates are applied throughout the network.

5.3 Gated Residual

Although standard residual connections have been highly successful in improving optimization and gradient flow in deep neural networks, they assume that the full residual transformation should always be added to the identity shortcut. In practice, however, not every learned feature refinement may be equally useful at every layer or at every stage of training. This suggests that allowing the network to selectively regulate the contribution of the residual branch could provide additional flexibility and reduce unnecessary noise.

The gated residual variant introduces a learnable gating mechanism that adaptively controls the strength of the residual branch before it is added to the identity shortcut. Rather than always combining the input x and residual transformation $F(x)$ at full strength, the model computes a gate value $g(x)$ that determines how much of the residual signal should be passed forward. In this formulation, the gate is defined as a sigmoid-activated function of the input, allowing the network to produce values between 0 and 1 that scale the residual contribution.

Conceptually, this mechanism allows the network to decide when the learned residual transformation should be emphasized, reduced, or suppressed. When the gate value is close to 1, the block behaves similarly to a standard residual connection, allowing most of the transformed features to pass through. When the gate value is closer to 0, the residual contribution is dampened, causing the block to rely more heavily on the identity shortcut. In this sense, the gated residual block allows the model to adaptively balance identity preservation and feature transformation.

Importantly, this variant preserves the same shared convolutional structure used in the baseline model. Each block still consists of two 3×3 convolutional layers with 64 filters, followed by batch normalization and ReLU activations. As a result, any observed differences in performance or optimization behavior can be attributed specifically to the gated routing mechanism rather than other architectural changes.

The gated residual block can be expressed as

$$y = x + g(x)F(x) \quad (3)$$

where x represents the input feature map, $F(x)$ denotes the learned residual

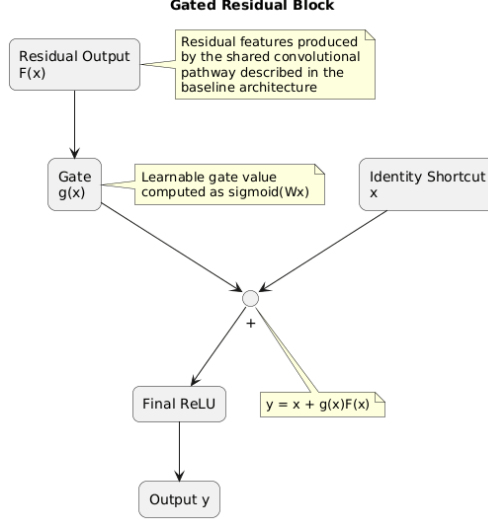


Figure 4: **Gated residual block.** The residual output $F(x)$ is multiplied by a learnable gate value $g(x)$ before being added to the identity shortcut x . The convolutional pathway producing $F(x)$ follows the same structure used in the baseline architecture.

transformation, and $g(x)$ is the gate value computed from the input. In this study, the gate is defined as

$$g(x) = \sigma(Wx) \quad (4)$$

where W is a learnable linear transformation and σ is the sigmoid activation function. Since the sigmoid outputs values between 0 and 1, the gate can continuously regulate the magnitude of the residual contribution rather than making a strictly binary decision.

The primary hypothesis behind this variant is that adaptive gating may improve training stability and representational efficiency by allowing the network to suppress less useful residual updates while preserving informative ones. This may be especially beneficial in deeper configurations, where some residual transformations may contribute noise or redundancy rather than meaningful refinement.

Because the identity shortcut remains intact, the gated residual architecture is still expected to preserve the healthy gradient flow characteristic of residual networks. At the same time, the learned gate values may provide additional interpretability by revealing how strongly the model relies on residual transformations at different layers. Overall, this variant is expected to perform comparably to or slightly better than the baseline residual model while offering greater flexibility in how residual information is routed through the network.

6 Experimentation

This section outlines the shared experimental framework used to evaluate and compare four variants of residual network architectures. All variants are trained and evaluated under identical conditions to ensure that any observed differences in performance can be attributed solely to the architectural changes being studied and not to differences in training setup.

6.1 Core Concepts

Before describing the experiment, we briefly define the key concepts that appear throughout this section.

Epoch — one complete pass through the entire training dataset. All variants are trained for 200 epochs. **SGD (Stochastic Gradient Descent)** — the learning algorithm used to update the network’s internal weights based on its mistakes after each pass through the data.

Learning Rate (lr) — controls how large of a step the model takes when correcting itself. A higher learning rate means larger corrections; a lower one means finer adjustments.

Momentum — causes the optimizer to carry forward a portion of its previous update direction, smoothing out corrections rather than reacting purely to the current step.

Weight Decay — a small penalty applied at each update to prevent any individual weight from growing too large, encouraging the model to stay well-behaved.

StepLR Scheduler — a plan that automatically reduces the learning rate at predetermined epochs, allowing the model to make approximate adjustments early in training and precise ones later.

Conv2d (2D Convolution) — a layer that scans the input with small filters to detect local patterns such as edges, textures, and shapes.

BatchNorm (Batch Normalization) — normalizes the output of a layer during training to stabilize learning and prevent values from growing uncontrollably.

ReLU (Rectified Linear Unit) — an activation function that sets any negative value to zero, introducing the non-linearity the network needs to learn complex patterns.

Residual Connection — a shortcut that adds a block’s original input back to its transformed output, helping gradients flow more easily through deep networks and preventing performance from degrading with depth.

Gradient Norm — a single number summarizing how large a layer’s weight adjustments are after a backward pass, used here to monitor how well gradients are flowing through each layer during training.

6.2 Architectural Requirements

All four variants will share the exact same foundation to ensure ”apples-to-apples” comparison.

- **Dataset:** CIFAR-10 (32 X 32 images, 10 classes).
- **Depths:** Every variant must be implemented and tested in two configurations: a 4-layer version and an 8-layer version.
- **Macro-Architecture:** Input Conv $\rightarrow$ N Blocks $\rightarrow$ Global Average Pool $\rightarrow$ Linear Classifier (10 classes).

6.2.1 Block-Level Structure

Every single block, regardless of the variant, must begin with this exact sequence (Using 3 x 3 convolutions with 64 filters).

- Input (x)
- Conv2d
- BatchNorm
- ReLU
- Conv2d
- BatchNorm $\rightarrow$ Output is $F(x)$

6.2.2 Variant-Specific Rules

Then, we proceed with variant specific rule implementation

- **Baseline:** $y = F(x) + x$
- **Scaled:** $y = x + \alpha F(x)$ (where α is a learnable parameter initialized to 1.0)
- **Gated:** $y = x + g(x) F(x)$ (where $g(x) = (Wx)$)
- **Plain (Ablation):** $y = F(x)$ (strictly no additions, multiplications, or skip connections)

6.2.3 Final Step

Pass y through a final ReLU activation before sending it to the next block.

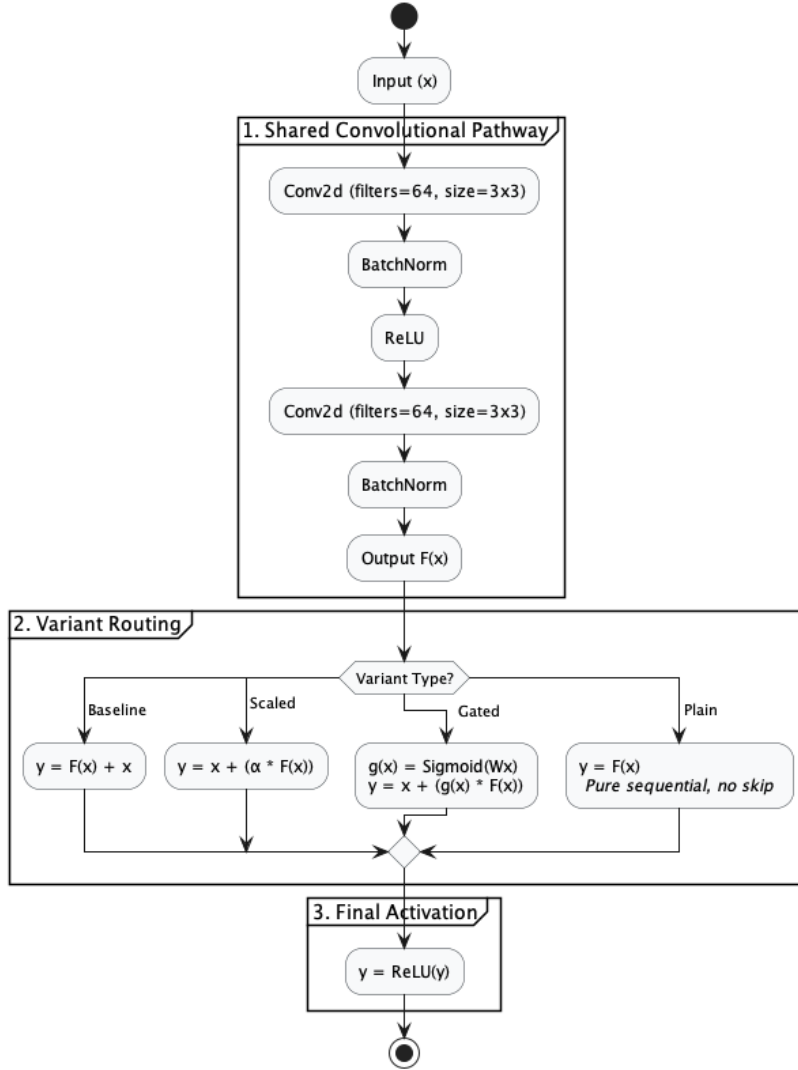


Figure 5: Shared Conceptual Block.

6.2.4 Optimizer & Training Settings

To ensure that the differences we observe are due to the architecture and not the training loop, the optimizer settings are strictly locked

- Total Epochs: 200 (200 complete passes through CIFAR-10).

- Algorithm: SGD (Stochastic Gradient Descent).
- Base Learning Rate (lr): 0.1
- Momentum: 0.9
- Weight Decay: $1 \cdot 10^{-4}$ (0.0001)
- Learning Rate Scheduler: StepLR
 - Epoch 1-100: lr = 0.1
 - Epoch 101-150: lr = 0.01 (divided by 10)
 - Epoch 151-200: lr = 0.001 (divided by 10 again)
- Data Augmentation:
 - Random Crop (pad by 4)
 - Random Horizontal Flip
 - Normalize (using standard CIFAR-10 mean and standard deviation)

6.2.5 Required Measurement

At the end of every epoch (200 times total per run), the following metrics must be recorded for all models:

- Training Metrics: Loss and Accuracy.
- Validation Metrics: Loss and Accuracy.
- Gradient Flow: The L2 norm of the gradients for each Conv2d layer after the backward pass.
 - Record 1: The mean gradient norm across all Conv2d layers.
 - Record 2: The specific gradient norm for each individual layer (required to generate "Gradient Norm vs. Layer Depth" plots).

6.3 Experimental Model

To evaluate the optimization benefits of different skip connection topologies, all four variants are subjected to an identical experimental protocol. Each architecture is instantiated in both 4-layer and 8-layer configurations, initialized with fixed random seeds to isolate structural routing effects, and trained in CIFAR-10 [6] for 200 epochs. The comparative analysis is based on three metrics: **top-1 classification accuracy**, **learning dynamics of training and validation**, and **layer-wise gradient flow**.

Classification accuracy serves as the primary metric for quantifying the degradation problem and assessing the mitigating effect of each residual variant. Although greater depth should theoretically expand the representational

capacity, the plain 8-layer network is expected to **underperform** its 4-layer counterpart due to degradation. On the other hand, the **Baseline**, **Scaled**, and **Gated** residual models are expected to overcome this by leveraging skip connections, enabling the deeper configurations to achieve superior or comparable performance to the shallower ones.

Beyond accuracy, training and validation curves reveal the convergence behavior of each architecture. Plain networks are expected to struggle to learn effectively, stalling early in training — a direct consequence of the degradation problem. By contrast, residual variants should converge more smoothly and stably. Among these, the Scaled and Gated models may exhibit more stabilized early-stage learning, as their dynamic regulation of residual update magnitudes allows better adaptation to training variations than the fixed Baseline residual connection.

Finally, to diagnose the root cause of these optimization behaviors, gradient magnitudes are tracked layer-by-layer during backpropagation. By recording the average gradient size at each convolutional layer per epoch, gradient strength can be plotted against network depth. Plain networks are expected to show a sharp drop in gradient magnitude as signals propagate backward through layers, especially in the 8-layer configuration. Tracking these values across the Baseline, Scaled, and Gated networks will reveal how effectively each skip connection mechanism preserves gradient strength with depth, providing a direct basis for comparing whether dynamic scaling or gating offers any advantage over the fixed Baseline residual connection.

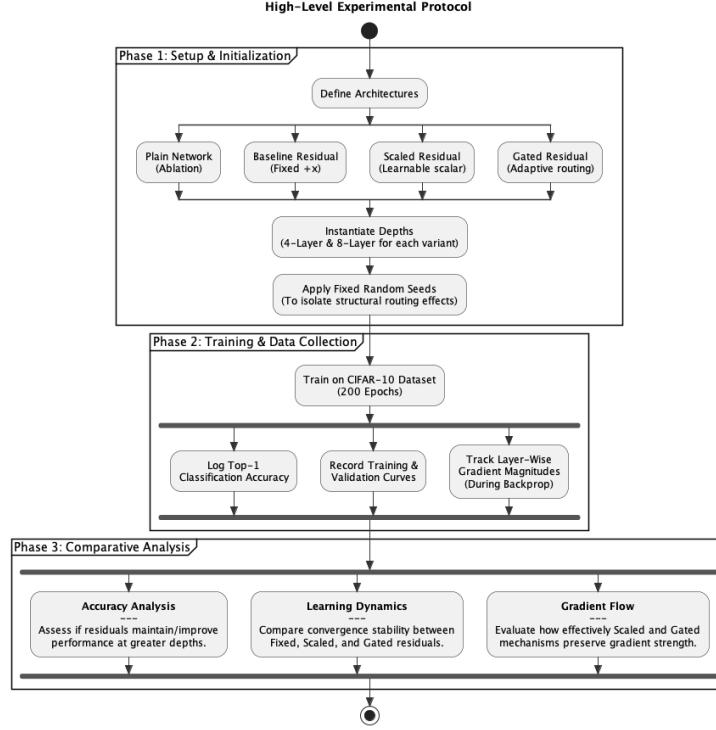


Figure 6: **Experimental Model.**

7 Timeline

7.1 Week 1

Goal: Set up the foundational code that all variants will share and implement the two most straightforward architectures. Both the Baseline and Plain networks complete a "dummy" 5-epoch run, successfully logging training loss, validation accuracy, and gradient norms to a shared file format (e.g., CSV).

7.1.1 Environmental Setup

- Initialize the Git repository.
- Agree on the Python framework and version (e.g., PyTorch).

7.1.2 Data Pipeline

- Set up CIFAR-10 integration:
 - Configure data loaders.

- * Apply the agreed-upon augmentations:
 - Random crop (pad by 4)
 - Random horizontal flip
 - Standard normalization

7.1.3 Shared Training Loop

- Build the main training script.
- Implement locked optimizer settings: **SGD**, **lr = 0.1**, **momentum = 0.9**, **weight decay=1e-4**
- Implement the StepLR scheduler (dropping at epochs 100 and 150).

7.1.4 Gradient Logging

- Implement hooks to intercept and calculate the average **L2 gradient norms** at each Conv2d layer during the backward pass.

7.1.5 Baseline Residual Network

- Implement in 4- and 8-layer depths.
- Ensure that the skip connection $\mathbf{y} = \mathbf{F}(\mathbf{x}) + \mathbf{x}$ is mathematically sound.

7.1.6 No Residual (Plain) Network

- Implement the plain (non-residual) architecture.
- Act as the primary tester for the gradient logging hooks — capturing vanishing gradients here is critical for the ablation study.

7.2 Week 2

Goal: All four models are integrated and ready for full-scale training.

7.2.1 Scaled Residual Network

- Implement the Scaled Residual network:
 - Ensure that the learnable scalar α is properly initialized to **1.0**.
 - Ensure that it is registered as a model parameter and updates correctly during backpropagation.

7.2.2 Gated Residual Network

- Implement the Gated Residual network:
 - Build the custom sigmoid gate mechanism $\mathbf{g}(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x})$.
 - Ensure that the matrix multiplication aligns with the tensor dimensions of the 64-filter 3×3 convolutions.

7.2.3 Code Integration

- Merge all four variants into the main branch.
- Refactor the code so that launching an experiment is as simple as passing a few arguments (e.g., `—variant gated —depth 8`).

7.2.4 Sanity Checks

- Run a single-batch overfit test on all four variants to confirm losses drop to near-zero, verifying that there are no detached computational graphs.

7.3 Week 3

Goal: Completely hands-off the experimental grid and collect all data required for the final analysis.

- Raw data files for accuracy, loss, and gradient norms are available to all team members.
- All 4 models have completed their 200-epoch runs.

7.3.1 Execution

- Launch the 8 official training jobs (4 variants $\times$ 2 depths).
- Each job must run for the full **200 epochs** using the locked hyperparameters.

7.3.2 Monitoring

- Monitor the first 10–20 epochs to ensure that the models do not diverge.
- Verify that the learning rate drops precisely at epochs **100** and **150**.

7.3.3 Data Archiving

- Once training is complete, safely store the following in a shared, backed-up drive
 - Logs: accuracy/loss arrays and layer-wise gradient norms.
 - Final model weights (.pt or .pth files).

7.4 Week 4

Goal: Translate raw logs into visual insights, analyze degradation and optimization behaviors, and finalize the research paper.

- The final research report is fully drafted, and ready for submission.

7.4.1 Documentation

- Generate training/validation curves and gradient norm visualizations for your assigned variant.
- Write the corresponding “Results & Analysis” subsection for your model.

7.4.2 Cross-Comparison

- Compile the final empirical summary.
- Compare the gradient decay of the 8-layer plain network with the residual pathways of the baseline, scaled, and gated variants.
- Construct the master comparison table covering: **Accuracy**, **Convergence Epoch**, and **Computational Overhead**.

7.4.3 Collaborative Review

- Read through the entire document to ensure a consistent “one voice” throughout.
- Verify that formatting, academic tone, and key terminology (e.g., “degradation problem,” “learning dynamics”) are used consistently.

7.4.4 Formatting

- Finalize all architecture diagrams (PlantUML).
- Ensure all diagrams are appropriately labeled and correctly referenced in the text.

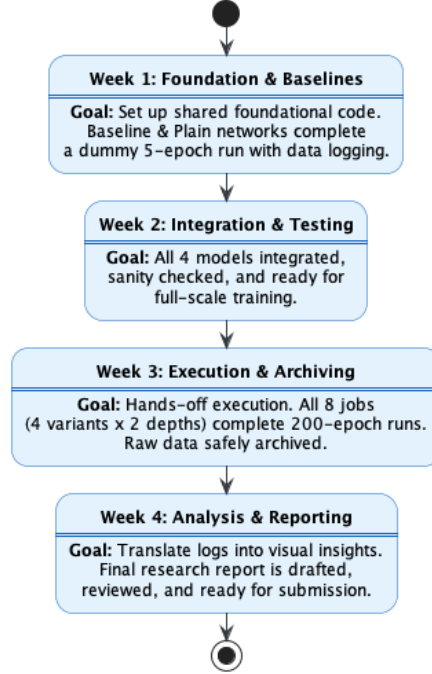


Figure 7: **High Level Timeline.**

8 Acknowledgments

- This paper was completed with the complimentary help of Gemini LLM
- Plant UML

[2]

References

- [1] Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473 (2014)
- [2] Gemini Team, Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A.M., Hauth, A., et al.: Gemini: A family of highly capable multimodal models (2023), <https://arxiv.org/abs/2312.11805>
- [3] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778. IEEE (2016). <https://doi.org/10.1109/CVPR.2016.90>

- [4] Hochreiter, S.: The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* **6**(02), 107–116 (1998)
- [5] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *International Conference on Machine Learning (ICML)*. pp. 448–456. PMLR (2015)
- [6] Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical report, University of Toronto (2009), <http://www.cs.toronto.edu/~kriz/cifar.html>
- [7] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 4510–4520 (2018)
- [8] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014)
- [9] Srivastava, R.K., Greff, K., Schmidhuber, J.: Highway networks. *arXiv preprint arXiv:1505.00387* (2015)
- [10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 30 (2017)